5. Ad-hoc approach: solver-in-the-loop

Following section 3.2.3, we develop solver-in-the-loop models for different MHD type problems.

5.1. Considered model: Time independent, divergence-free CNN

The model considered throughout this section consists of a simple CNN with inductive biases motivated by physical and simulation constraints. Depending on the solver mode, finite volume or finite difference, the input state contains either 8 or 11 channels. The state is passed through the network, which consists of a series of convolutional layers intertwined with activation functions. Afterwards, the state is lifted again to the original number of channels. Further, the magnetic field correction is transformed by applying the curl operation, ensuring the magnetic field divergence is 0. The correction is then multiplied by the timestep, making the model time independent by learning the time derivative of the correction. Finally, the resulting correction is added to the initial state. A scheme of the model can be seen in Figure 5.1.

Commenting on the solver modes, Astronomix implements both finite difference and volume. Depending on which is used, the model acts in slightly different ways:

- **Finite volume**: in the finite volume mode, 8 channels are used (density, pressure, 3 velocity fields and 3 magnetic fields), and all of them are used during the correction and loss calculation.
- **Finite difference**: in the finite difference mode, Astronomix saves an extra vector field to the state. This vector field stores the intermediate cell values to compute fluxes between differences later. The field is obtained from the magnetic field centre values; therefore, the intermediate fluxes are neglected when computing the correction and are afterwards computed from the magnetic field centre values. Equally, the fluxes are not considered when computing the losses.

The CNN model simplicity is deliberate; in this fashion, we minimise the performance decrease of the simulation. More complex architectures could be considered, but a

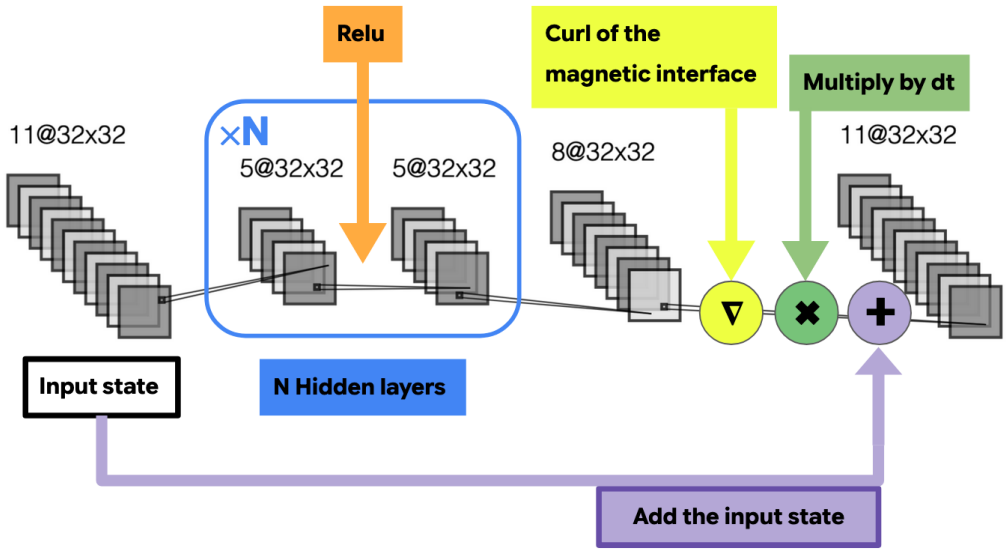


Figure 5.1: The model considered for solver-in-the-loop applications. The model predicts a divergence-free and time independent correction to the state by computing the curl of the magnetic field correction and predicting the time derivative of the correction.

thorough evaluation of their performance is beyond the scope of this work. It is also important to note that the inductive biases introduced into the model reduce its expressiveness [Cazenavette and Lucey, 2021]. In this case, three constraints are imposed on the space of possible corrections. First, the magnetic field correction is computed as the curl of the network output, ensuring that it is divergence-free. Second, ReLU activation functions are used for the density and pressure fields, enforcing positivity and preventing nonphysical negative values. Finally, predicting the correction time derivative further constrains the space of possible corrections. Nevertheless, considering the scale of the network in this case, the biases encode key information not only for the network but also to ensure the simulations' stability.

5.2. Training pipeline and stability mechanisms

Following the notation introduced in section 3.2.3, we described the basic training procedure used in the following sections. Commenting first on the technical aspects, the

simulation runs in JAX, and its API allows for the gradients and models to be fully written in JAX aswell. Nevertheless, a series of wrappers which implement neural network capabilities with similar APIs to those of established neural network libraries such as PyTorch exist, examples of such wrappers are Haiku [Hennigan et al., 2020] (which was replaced by Flax [Heek et al., 2024], both developed by Google) or Equinox [Kidger and Garcia, 2021]. Out of these, we use Equinox, which implements PyTorch like APIs and advanced techniques to manage PyTrees.

Given an initial state, a high resolution simulation is computed that will be used as a reference. The downscaling transformation τ is then applied to both the initial state $\tau(s_{t=0|HR}) = s_{t=0|LR}$ and the reference states $\tau(s_{t=T|HR}) = r_{t=T}$. Then, during training, the low resolution initial state is used to initiate the solver-in-the-loop enhanced simulation, thus obtaining the final low resolution enhanced state $\tilde{s}_{t=T|LR}$. The loss is then computed between the reference state and the final low resolution enhanced state. Exploiting now the differentiable nature of the simulation, the loss is backpropagated through the simulation to obtain the gradients with respect to the network parameters. This procedure is showcased in Figure 5.2.

One important caveat of training SOL models is ensuring the stability and physicallity throughout the simulation. Since any unphysical realisation of the network during training invalidates the corresponding training run, several measures are taken to promote stable training. The architecture used, introduced in the previous section, promotes stability by incorporating safeguards and inductive biases. Still, gradients can explode and lead the simulation quickly to unphysical results. Hence, the model has to be initialised with rather small weights, in our case, we scale the initial weights (typically randomly assigned) by a factor of 0.02. In addition, the learning rate scheduler follows an initial linear warm up followed by cosine decay to ensure a well-posed and stable loss environment. Lastly, we apply extra noise to the initial state to ensure regularisation. Nevertheless, most of the training stability depends on the problem chosen to train on, for instance, noise-based regularisation strategies are not suitable for turbulent setups due to their chaotic nature.

A normalised mean squared error is used as the de facto loss for the following experi-

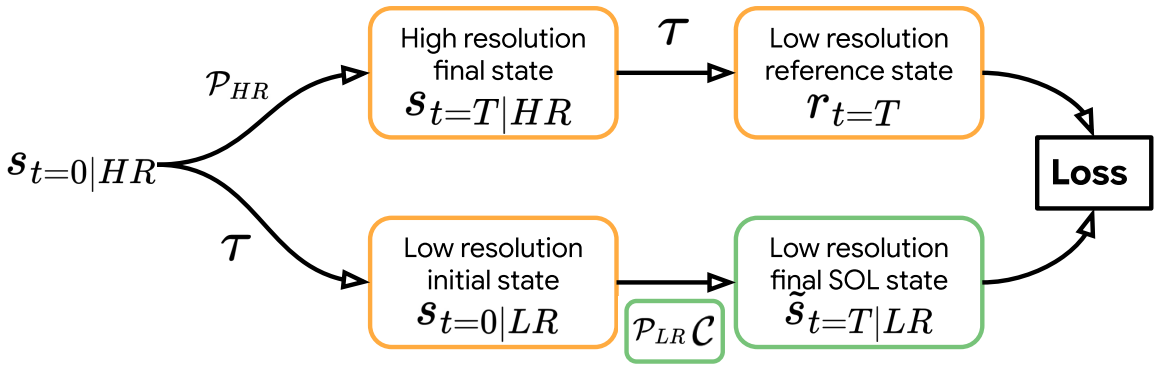


Figure 5.2: solver-in-the-loop training procedure. An initial high resolution state $s_{t=0|\mathrm{HR}}$ is both propagated forward in time $\mathcal{P}^n_{HR}(s_{t=0|\mathrm{HR}}) = s_{t=T|\mathrm{HR}}$ and downscaled to a lower resolution $\tau(s_{t=0|\mathrm{HR}}) = s_{t=0|\mathrm{LR}}$. The final high resolution state is then downscaled and used as the reference state during the loss calculation $\tau(s_{t=T|\mathrm{HR}}) = r_{t=T}$. Finally the initial low resolution scale is used to initiate the low resolution SOL corrected simulation $(\mathcal{PC})^n_{\mathrm{LR}}(s_{t=0|\mathrm{LR}}) = \tilde{s}_{t=T|\mathrm{LR}}$ and the loss is computed and backpropagated to optimise the network weights.

ments. The normalisation is done on a per state basis using the reference state. In our experiments, we tried other losses, such as physically informed losses like total energy losses or, in particular for turbulence, energy spectra or vorticity-based losses. However these alternatives didn't improve the model's performance due to training instabilities and complexity. For multiproblem experiment setups, the loss formulation was kept simple. As each of the initial states, when run through the training pipeline, returns a gradient, thus making this a multiobjective problem already. Further studies on losses with solver-in-the-loop models are needed.

5.3. Multiproblem API

We develop an API that supports iterative model training across multiple problems and facilitates the implementation of different type problems with custom parameters. The API works by defining an abstract class `BaseProblem`, which specifies the common interface that every simulation problem must implement. It defines the following properties and methods:

- `name` (abstract property): a property returning the unique name of the problem, used to identify it within the problem catalogue.
- `t_end` (abstract property): a property specifying the final simulation time.
- `generate_initial_state(SimulationConfig, SimulationParams)` (abstract method): generates the initial simulation state together with the corresponding simulation configuration, parameters, and registered variables.
- `get_hyperparams()` (abstract method): returns a dictionary containing the problem-specific hyperparameters used for configuration and deterministic naming.
- `filename` (computed property): returns a deterministic filename from the problem name and parameters, allowing for unique identification of each problem's instance.

Each problem instance is then represented by a `ProblemDescriptor` object, which stores its name, parameters and other fields that facilitate easy training and iterative experimental design. The `ProblemDescriptors` objects are aggregated into a list, which is later used to define an instance of the `ProblemManager` class. `ProblemManager` generates the high resolution - low resolution training pairs, provides methods for storing and loading lists of `ProblemDescriptor`, obtains evaluation data and allows for random problem sampling.

The `BaseProblems` that will be used in the following are:

- MHD Blast: following section 3.3.1, the parameters that define a blast instance in the API are `B_0` and `B_direction` which determine the magnitude and direction of the magnetic field, and `r_0` which defines the radius of the high pressure sphere at the centre of the simulation box. In the following, we characterise `B_direction` by the spherical coordinate angles (θ, ϕ) as the code normalises the vector.
- OT vortex: following section 3.3.2, the parameters that determines the OT vortex are ϵ, which defines the perturbation in the field perpendicular to the vortex plane, `vortex_axis`, which defines the axis perpendicular to the vortex plane, and `parity`, which defines the rotation direction of the vortex.

Each problem considered during training returns an optimisation gradient. Therefore, when considering multiple problems within an epoch, the task becomes a multiobjective, which requires gradient accumulation strategies. In the following, we use the conFIG multiobjective method [Liu et al., 2025], which ensures conflict-free gradient updates via positivity enforcement of the dot product between the final gradient update and each of the individual gradients.

5.4. Experimental evaluation

In this section, we present the experimental evaluation of solvers-in-the-loop across five setups: a single field output baseline, a temporal generalization study using curriculum learning, and three multiproblem experiments consisting of a dual-problem setup, a rotation-generalization study and a six-problem suite.

5.4.1. Single field output model

In this experiment a single field correcting SOL model is trained and further optimised. This experiment was initially designed as a lightweight SOL test, thus low and high resolutions 16^3 and 32^3 are used, further in the following 4 experiments the resolutions are increased. A single problem is used to train on: an MHD blast with the same specifications as in 3.3.1: $\texttt{B\_0} = 10.0$, $\theta = \pi/2$, $\phi = \pi/4$ and $\texttt{r\_0}= 0.125$. Noise injection is applied at the initial state for regularisation. As a learning rate schedule, we use the cosine warmup learning rate with an initial value of 8.5×10^{-5}, a peak learning rate value of 4×10^{-3} and an ending value of 3×10^{-5}, with the warm up regime occupying 25% of the training. The target state is defined at $t_{end} = 0.2$.

We obtain the best hyperparameter combination by running an Optuna optimisation with 100 studies. The best objective results found are a magnetic field only output model with 24 hidden channels and 2 hidden layers. The training loss evolution can be seen in Figure 5.3. In Figure 5.4 the diagonal profiles at the training target time can be seen. The evolution of the MSE over simulation time can be seen in figure 5.5, here two different phenomena can be observed:

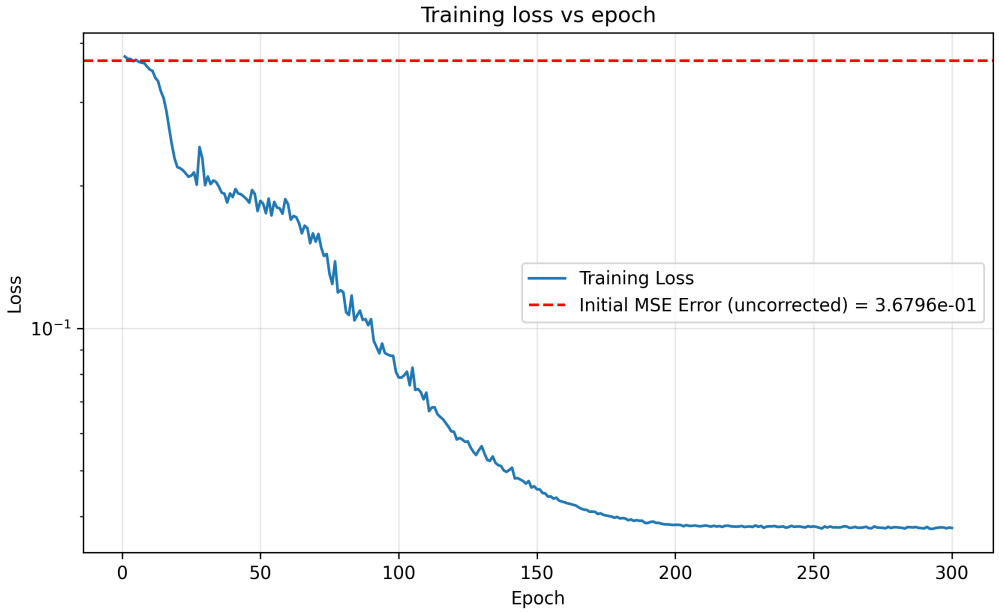


Figure 5.3: Loss epoch evolution of the magnetic field only output model trained on a single MHD blast initial state with low resolution 32^3 and high resolution 64^3.

- Observing the dashed red line, which is the error of the low resolution simulation (without solver-in-the-loop) when initialized from the last state used to train the SOL model ($t = 0.2$). It shows a better performance throughout the simulation than the SOL-enhanced simulation. This suggests that the SOL model does not generalise well onto unseen simulation states. This comparison is typically omitted from SOL simulation loss over time plots. However here the results highlight the including it of showing as it might reveal a lack of generalisation on to unseen simulation states.

- An abrupt increase in the error of the SOL simulation can be observed at the early stages of the simulation. One possible explanation is overfitting in both phase space and state space. Within the phase space, given a non-chaotic problem such as the MHD blast, where the initial state strongly determines later states, the model learned to modify mostly the initial state. In this case, the stability of the problem allows for such a procedure. Besides, as the model is trained on one single initial state, the early stages of the simulation, remain relatively similar across epochs.

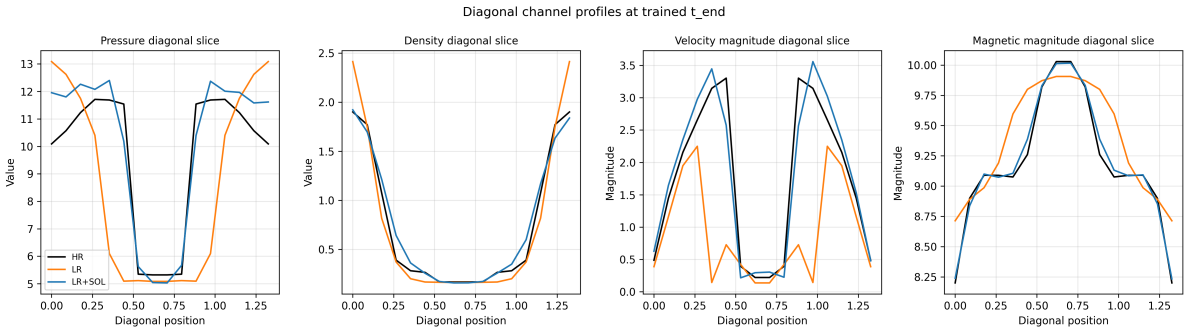


Figure 5.4: Diagonal profiles from the lower left corner to the upper right corner in the x,y field with a fixed z at the middle of the simulation box at target time $t = 0.2$. The black line is the high resolution state, the orange line is the low resolution and the blue line is the SOL enhanced low resolution. The model successfully captures the high resolution structures.

As a result these states are encountered more frequently by the network during training. Thus, it is the easiest path of optimisation for the network. Both these effects combined could explain the sharp peak at the beginning of the simulation. Hence, this signifies the need to augment the problem and initial state dataset pool, as well as other training strategies.

The model was trained on an Nvidia RTX 2080, and the whole training took 25000 seconds (7 hours), approximately 70 seconds per epoch. We note the time complexity that training such models entails, especially in 3D scenarios. Furthermore, the resolutions used (32^3 and 16^3) aren't representative of real scientific scenarios due to their lack of resolution and accuracy. Nevertheless, from the loss training evolution 5.3, it's apparent that an early stopper would have reduced the number of epochs to 250, however, the discussion still holds. We extract that solver-in-the-loop models are resource costly and need further memory and time optimisation techniques.

Following section 3.3.1 it was expected to obtain a model that prioritizes the magnetic field correction as the blast setup used was designed to test the magnetic field capabilities of the simulation, thus a large portion of the error originates from the unresolved magnetic field. In the following sections, we explore the same setup with full models to observe which other field (beyond the magnetic field) is corrected the most.

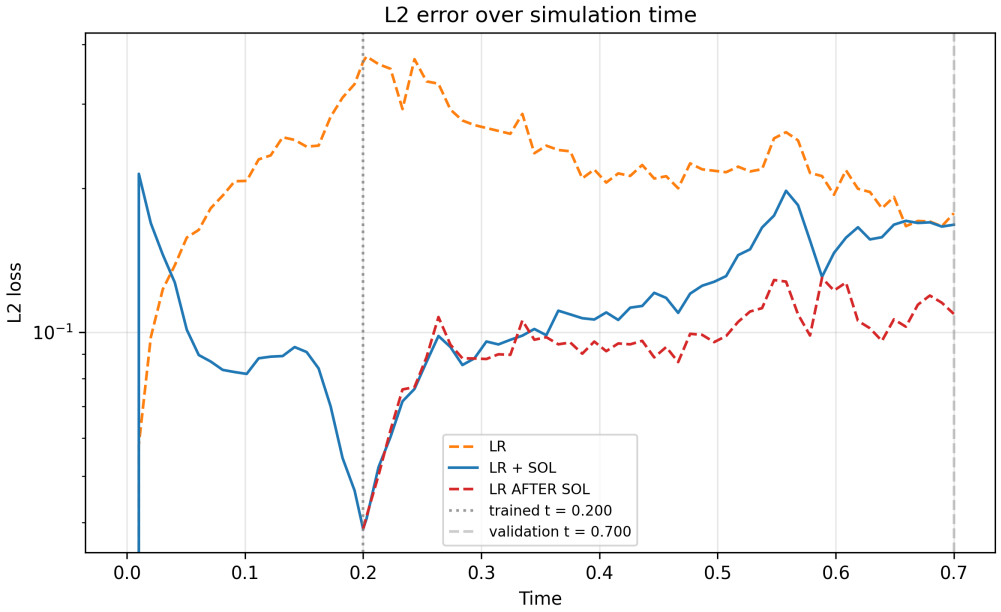


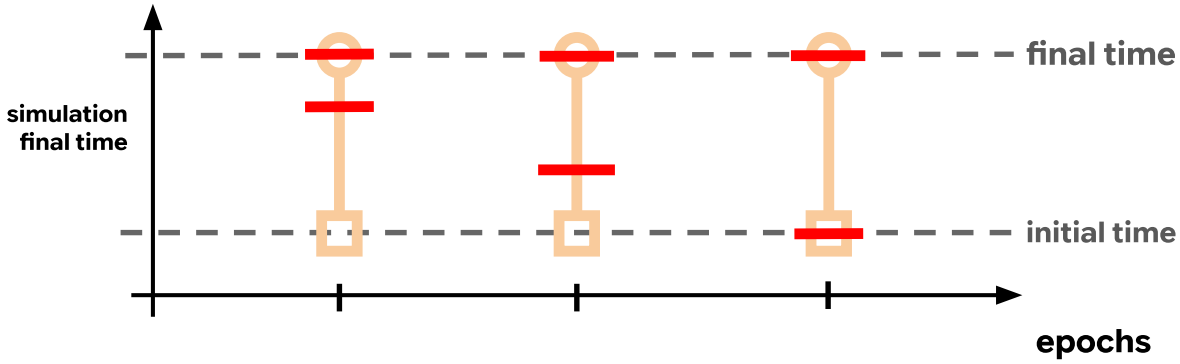
Figure 5.5: MSE error over simulation time of the magnetic field only output model for an MHD blast problem. The orange line is the error of the low resolution model, the blue line is the error of the low resolution SOL enhanced simulation, and the red line is calculated by taking as the initial state the last state at which the SOL was trained and simulating without the SOL. Hence, if the blue line goes over the red line, it signifies that the model is not able to generalise over unseen regimes.

5.4.2. Curriculum learning

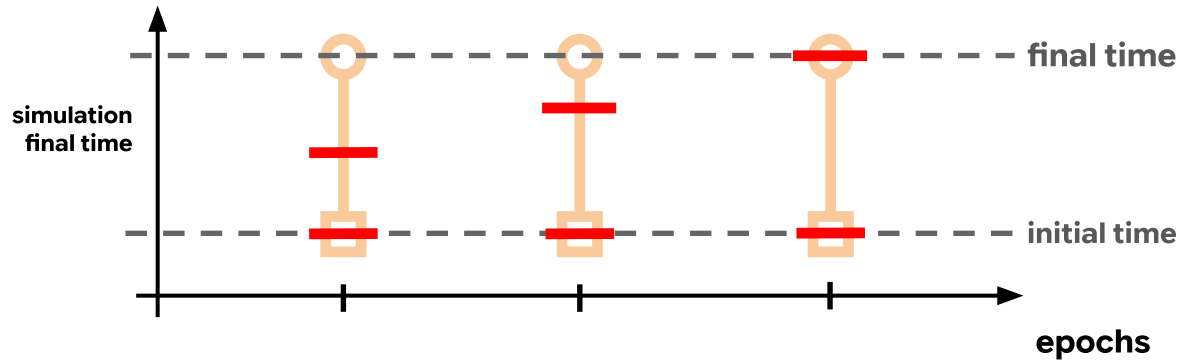
Curriculum learning is a machine learning technique in which the difficulty of the training set increases as the training progresses. To apply such a technique, a measure of "difficulty" is needed that works on a case by case basis. In our case, as the problem we are training on (MHD Blast) is rather stable, we are going to increase the simulation time as the training progresses. This can be done in 2 fashions, which are represented in Figure 5.6:

- Front to Back: Keeping the initial time constant and moving the end time further away.
- Back to Front: Keeping the end time constant and moving the initial time earlier.

We train 3 models on an MHD Blast with low resolution and high resolution, 32^3 and 64^3.



(a) Back to Front curriculum learning approach. The end time is kept fixed and the initial time is moved earlier as the training progresses.



(b) Front to Back curriculum learning approach. The initial time is kept fixed and the final time is moved later as the training progresses.

Figure 5.6: Curriculum learning solver-in-the-loop approaches. The red lines signify the portion of the simulation used to train on.

These resolutions are twice of those used in the previous section, 16^3 and 32^3. Besides from the curriculum learning, the rest of the training setup is taken from the previous section, including learning rate values and problem setup. The different curriculum learning strategies used are:

- No curriculum learning (No CL): Our baseline model, training with the whole simulation length through the whole training.
- Front to Back (FTB): Split into 3 different regimes, with a fixed starting time and 3 different end times $t_{end} = (0.1, 0.15, 0.2)$ on these epochs $= (150, 150, 200)$

respectively.

- Back to front (BTF): Split into 3 different regimes, with a fixed end time and 3 different starting times $t_{initial} = (0.1, 0.05, 0.0)$ on these epochs $= (150, 150, 200)$ respectively.

The loss curves for each of the models can be seen in Figure 5.7, we kept the 500 total epochs and no early stopping for a more direct comparison. Using an early stopper, we would have seen an earlier convergence (in training time) of both the BTF and FTB, as when training with less simulation time the backpropagation step length is reduced significantly.

The error evolution over simulation time can be seen in Figure 5.8. Commenting first on the target point, we can see that both FTB and No CL outperform BTF, which signifies the reduced number of epochs the BTF saw the target time. Afterwards, we can look at the initial error. Interestingly the FTB has the lowest initial error introduced, as it saw the initial state for fewer epochs; in that way, the FTB poses as a good strategy to avoid the overfitting in phase space present in each of these models. Considering the error tail beyond the target time, we observe that the No CL model produces little to no correction of the simulation error. This is further supported by the model outputs shown in Figure 5.9, where the magnitude of the corrections produced by No CL is approximately one order of magnitude smaller than those produced by the FTB and BTF models. Consequently, while both BTF and FTB actively attempt to correct the simulation beyond the target time, these corrections worsen its accuracy, whereas the much smaller corrections from No CL have a negligible effect.

BTF and FTB pose as suitable strategies to circumvent overfitting, which is to be expected from the proposed solution of [Mikhaeil et al., 2022]. Where, in the case of chaotic problems for RNNs, it is proposed to sparsely substitute the inputs of the network for the real values while training. BTF and FTB are similar to this strategy with some caveats, we change the inputs in between epochs, whereas the proposed methodology involves changing the input state during the forward pass of the network. Besides, although the referenced work studies chaotic systems, here we can invert the logic:

instead of chaotic systems, we can think of RNNs overfitting to stable problems and apply the same solution, although in the case of curriculum learning this is done in a discretised fashion. Still, the No CL by avoiding to correct the state beyond the target time actually performs better on the unseen times than the 2 other models. Hence more regularization strategies are needed to achieve better generalization and performance across the complete time range.

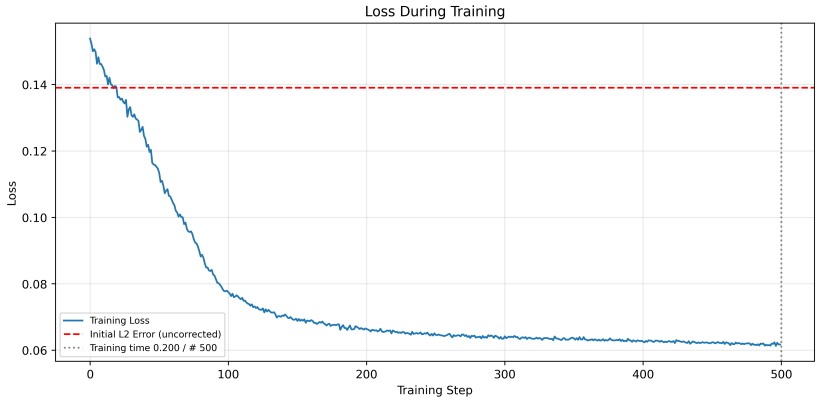
In Figure 5.9, the relative model output over simulation time is shown. The three models behave similarly, applying larger corrections early in the simulation, after which the magnitude of the model output remains relatively constant. Returning to the discussion in Section 5.4.1, where we studied a model predicting corrections for a single component, this section considers the full corrector model. This allows us to examine the relative importance of each field in the predicted correction.

When interpreting these results in the context of supernova simulations, however, we should exclude the magnetic field from this comparison. The MHD blast setup used in this section was specifically designed to test the simulation's magnetic field capabilities and therefore does not represent a typical supernova setup, in which the magnetic field is expected to play a comparatively smaller role. Including the magnetic field in this comparison would therefore give it a disproportionate importance relative to the other fields. Excluding it allows us to examine the relative importance of the hydrodynamic variables more representative of a supernova blast. With this consideration, the corrector is observed to prioritise the velocity components, suggesting that the velocity field is one of the most important fields when simulating supernova blasts. This is consistent with the role of velocity in transporting both energy and mass, meaning errors in the velocity field can propagate to the density and pressure fields.

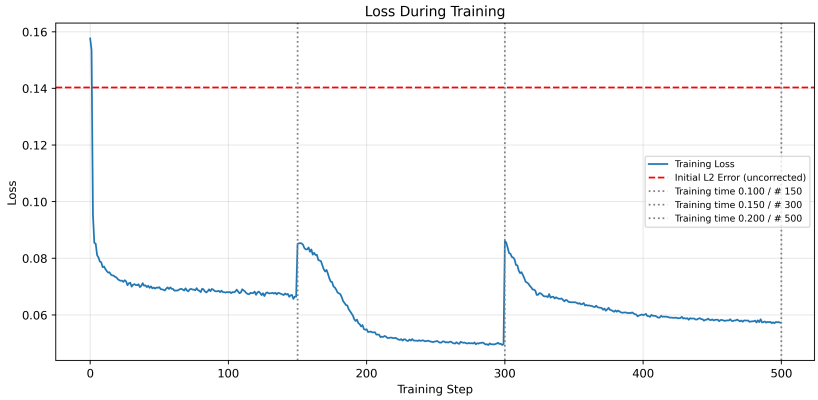
5.4.3. Biproblem multiproblem training

We train a model on the following two problem instances to test the multiproblem API and generalisation capabilities of a model trained on a reduced set of problems:

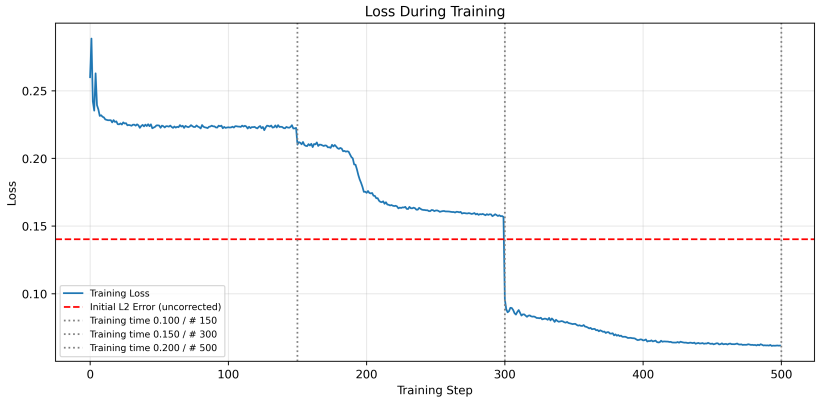
- **MHD blast**: With `B_0` $= 10$, $\theta = \pi/2$, $\phi = \pi/4$ and `r_0` $= 0.125$.



(a) Benchmark model loss evolution

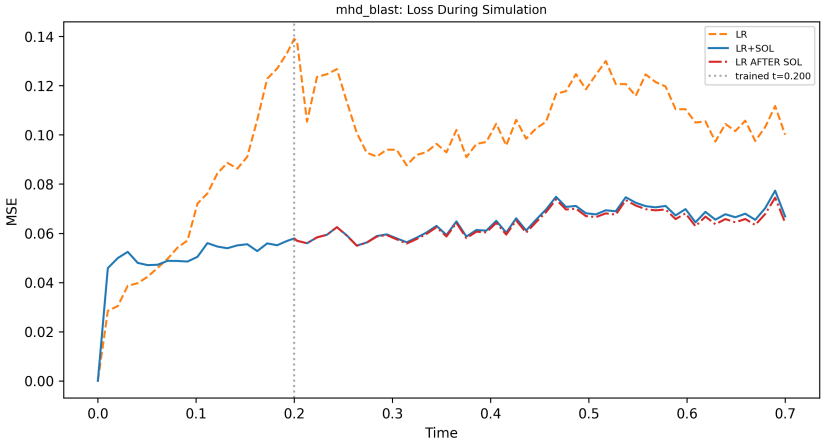


(b) Back-to-front model loss evolution

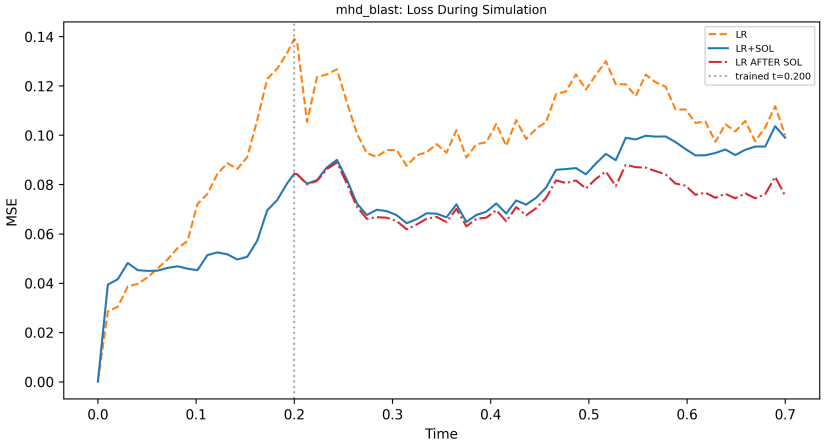


(c) Front-to-back model loss evolution

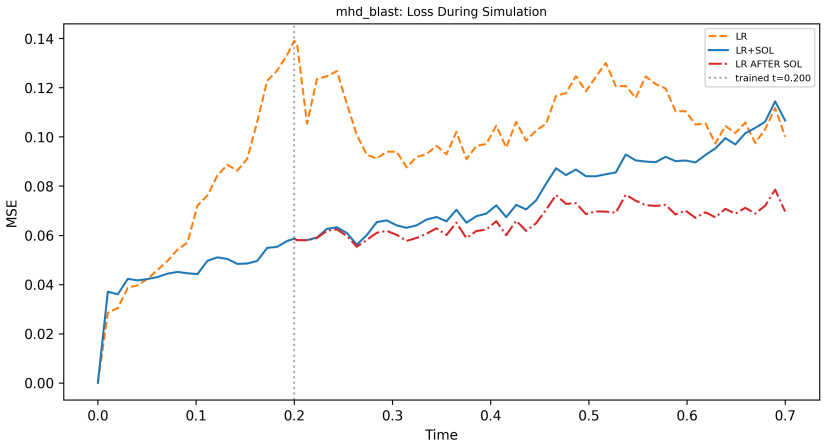
Figure 5.7: Training loss evolution for the different curriculum learning strategies.



(a) Benchmark model loss blast MSE

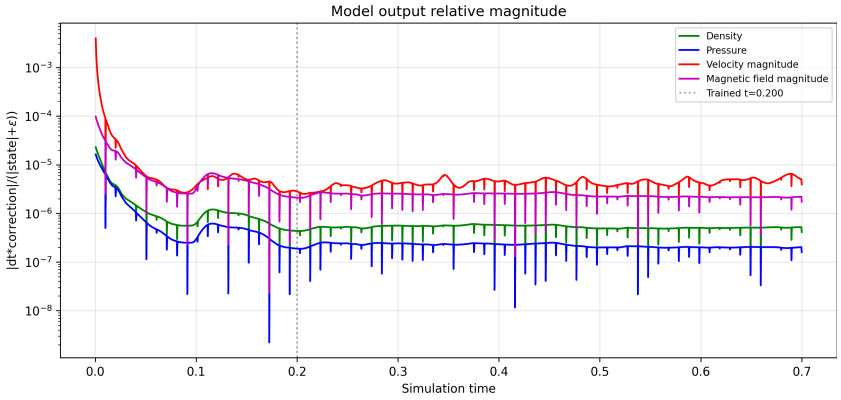


(b) Back-to-front model loss blast MSE

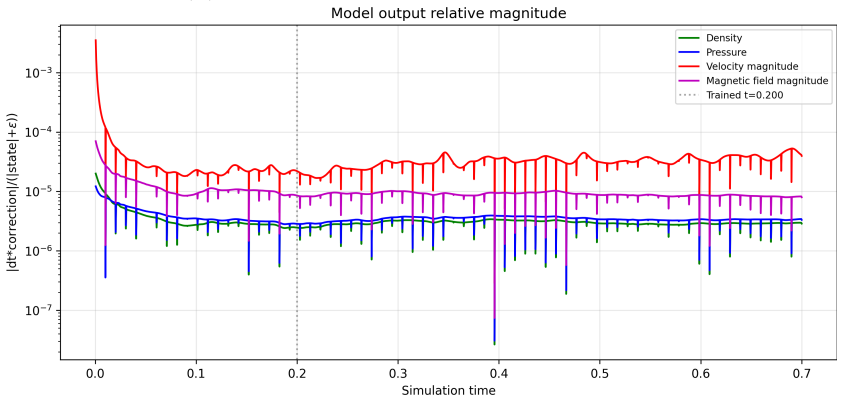


(c) Front-to-back model loss blast MSE

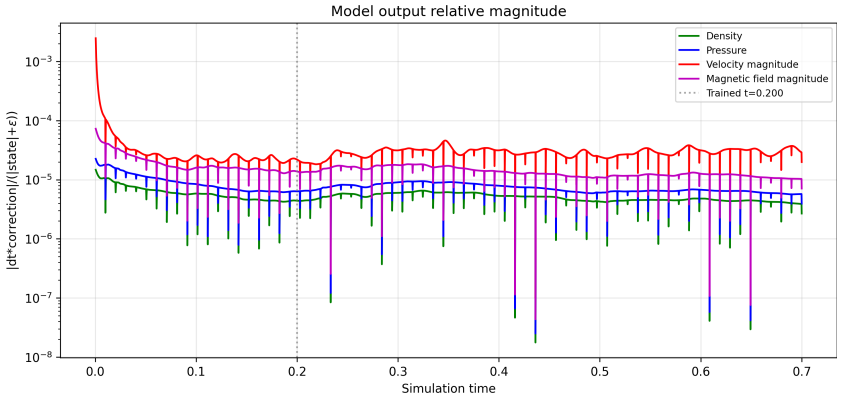
Figure 5.8: Simulation loss on an MHD blast for the different curriculum learning strategies.



(a) Benchmark model relative output



(b) Back-to-front model relative output



(c) Front-to-back model relative output

Figure 5.9: Relative output of each of the considered curriculum learning trained models. The metric presented is $|\Delta t \times \text{correction}|/(|\text{state}|+\epsilon)$ where ϵ is a small constant to ensure numerical stability. The sharp peaks are due to the adapatative time stepping and don't reflect the behaviour of the model.

- **OT vortex**: With $\epsilon = 0.2$, `vortex_axis` = z axis and `parity` $= -1$.

The training loss evolution is presented in Figure 5.10. Overall, we observe the model converge on the 2 problems simultaneously, showing that the network can improve on both without observable conflicts. The OT vortex objective is weakly dependent on the resolution when compared with the MHD blast, thus acting as a physical consistency objective for the model. This observation then suggests that the addition of weakly dependent resolution problems may help ensure physically self-consistent models while maintaining the same performance.

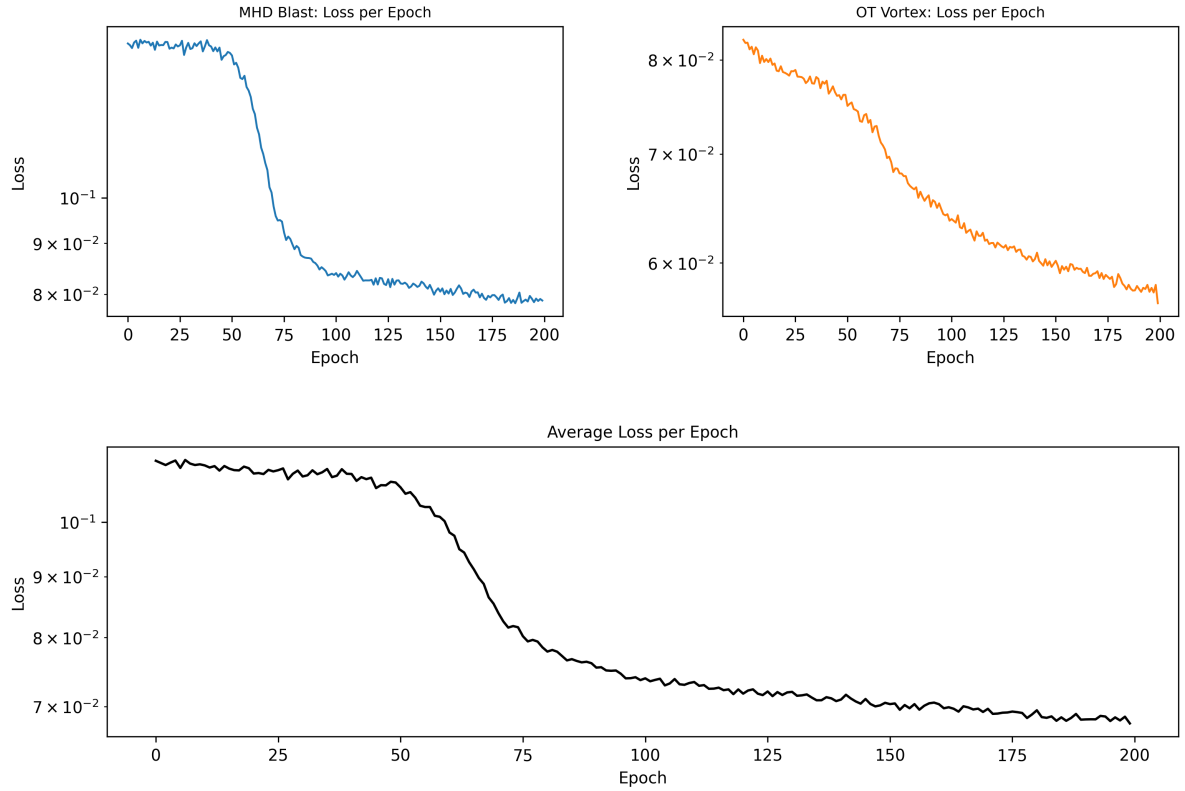
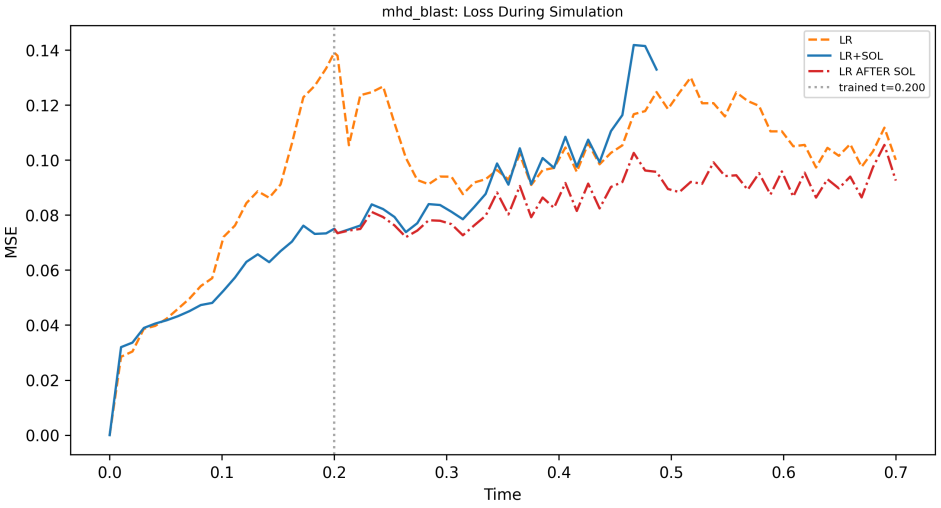
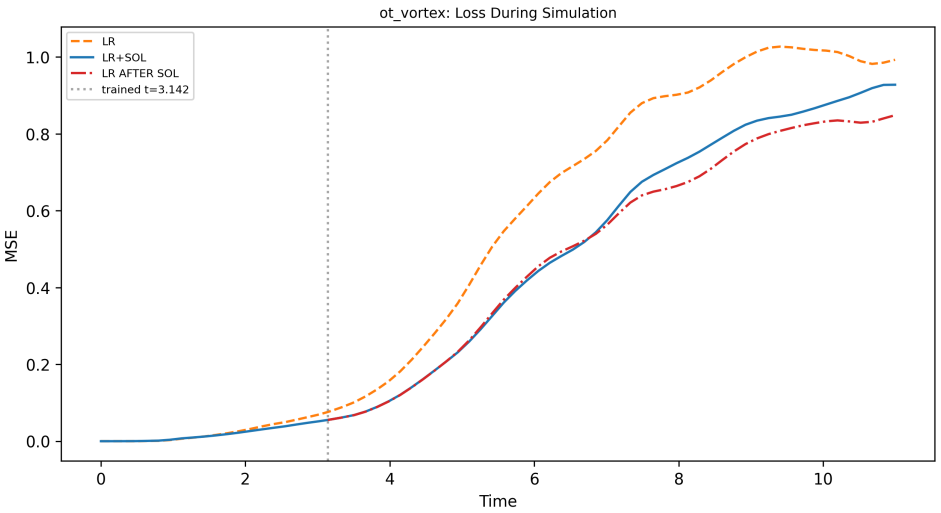


Figure 5.10: Loss training evolution over a set of 2 problems, an MHD blast and an OT vortex.

The loss against simulation time for both problems is presented in Figure 5.11. On the later part of the MHD blast problem the model quickly transforms the simulation unstable and crashes. In particular, the corrector predicts a non-physical correction, which, regardless of the safeguards imposed, runs the simulation rapidly into crashing.



(a) Loss simulation evolution over the MHD blast.



(b) Loss simulation evolution over the OT vortex.

Figure 5.11: Loss simulation evolution for the MHD blast and OT vortex. The MHD blast pure SOL run crashes during the simulation, thus, its line does not reach the far right end.

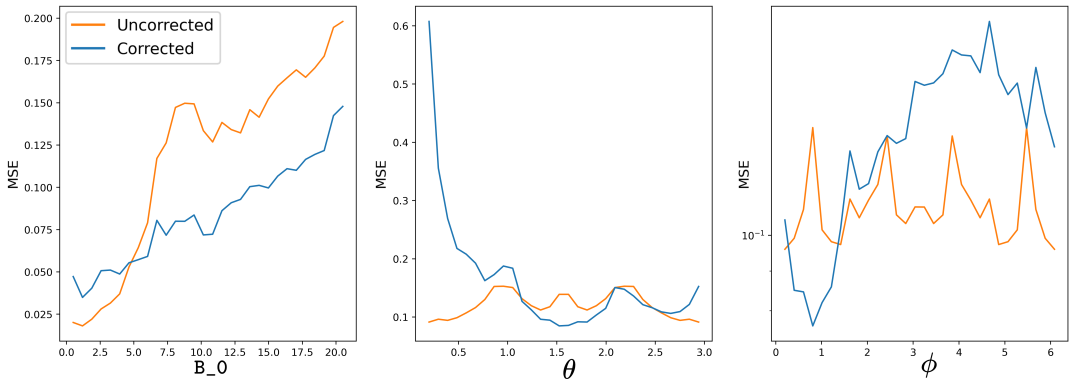
Therefore, although the addition of the OT vortex problem did not prevent loss convergence or stable training, the resulting final model is less robust during long simulations onto unseen phenomena. This may be attributed to several factors, including the limited number of effective training epochs compared with the previous model which was trained on 500 epochs, the small dataset of 2 problems, or the gradient aggregation strategy which promotes conflict-free updates while and might degrade the individual performance on each of the problems. Although the loss was still decreasing at the end of training, its rate of decrease had become relatively small. Consequently, extending the training could potentially provide further improvements, while the use of multiple curriculum learning strategies may have limited the effective training time available for learning the MHD blast dynamics.

Lastly, we examine how this model generalises over unseen parameters. We plot the MSE loss at the target time while varying the problems parameters. Although a more thorough examination would be needed to ensure real generalisation, the current performance is already too poor. Therefore, improving the performance at the target time is a prerequisite to continue looking further in a broader study. The figures are presented in 5.12. The improved performance appears only in the regions around the trained parameters, this phenomenon is extreme for the OT vortex where the only combination of parameters that appears to improve the baseline is the trained one. This observation, together with the lack of angle generalisation in the MHD blast plots, may be attributed to CNN models not being rotation invariant. A typical strategy to achieve rotation invariance is data augmentation, which we try in the following.

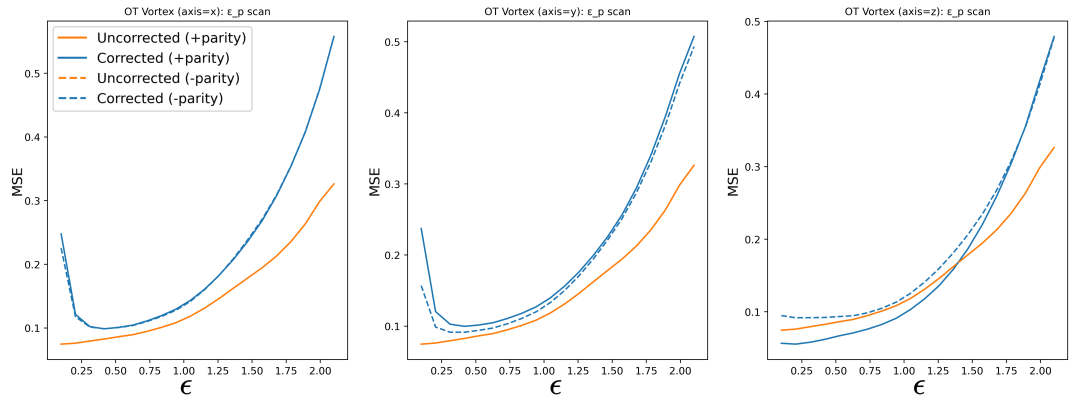
5.4.4. MHD Blast Azimuthal rotation generalization model

We train a model on the following 3 problem instances to test for angle generalisation of the CNN in the azimuthal direction:

- **MHD blast**: With `B_0` $= 10$, $\theta = 0.8$, $\phi = \pi/4$ and `r_0` $= 0.125$.
- **MHD blast**: With `B_0` $= 10$, $\theta = \pi/2$, $\phi = \pi/4$ and `r_0` $= 0.125$.
- **MHD blast**: With `B_0` $= 10$, $\theta = 2.3$, $\phi = \pi/4$ and `r_0` $= 0.125$.



(a) Loss parameter study of the MHD blast. The blast radius is fixed in all panels. From left to right, (i) the magnetic field strength is varied while the field direction is fixed at $\theta = \pi/2$ and $\phi = \pi/4$; (ii) the polar angle θ is varied while the magnetic field strength and azimuthal angle ($\phi = \pi/4$) are fixed; (iii) and the azimuthal angle ϕ is varied while the magnetic field strength and polar angle ($\theta = \pi/2$) are fixed, restricting the field direction to the x-y plane



(b) OT vortex parameter study. The ϵ parameter, which controls the perturbation amplitude in the plane perpendicular to the vortex axis, is varied. From left to right, different vortex orientations are studied by varying the axis perpendicular to the vortex plane, together with the effect of parity.

Figure 5.12: Generalization study of the model trained simultaneously on an MHD blast and an OT vortex. The parameter studies evaluate the model's ability to generalize to configurations beyond those encountered during training. The parameters used during training are $B_0 = 10$, $\theta = \pi/2$, and $\phi = \pi/4$ for the MHD blast, and `axis`=z, $\epsilon = 0.2$, and positive parity for the OT vortex.

The training loss evolution is presented in Figure 5.13. The model converges on the 3 problems simultaneously with no apparent issue. The generalisation over the parameters is presented in 5.14. In the middle plot, the study over θ can be seen; comparing it now with 5.12, the overall generalisation can be seen in work. Although overall performance decreased, especially when looking at the rightmost plot, the difference between the corrected loss and target loss at the trained point $\phi = \pi/2$ is reduced. Thus we can extract an overall tradeoff between performance and generalisation, which is to be expected, as before the model was overfitting the problem and now, through data augmentation, we are weakly enforcing biases such as rotation invariance. Of course this was done at a small scale, and larger tests are needed to ensure these claims.

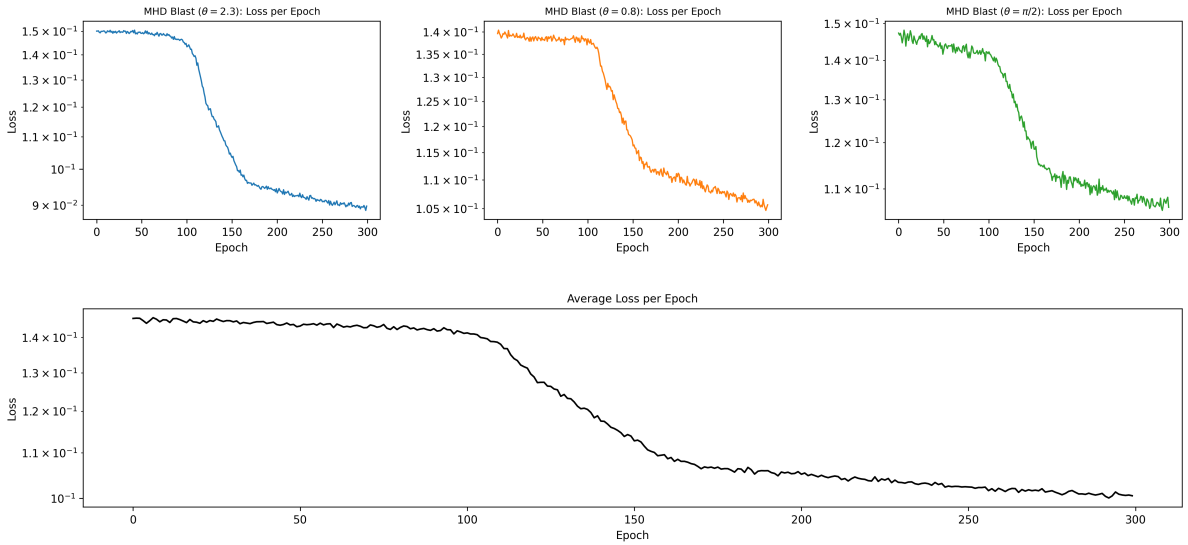


Figure 5.13: Training loss evolution of the MHD blast angle study model. The model was trained on 3 MHD Blasts with equal parameters but the azimuthal direction of the initial magnetic field (θ).

5.4.5. Scaled multiproblem training

As a final experiment, we train the model simultaneously on six problems, consisting of three MHD blasts and three OT vortices:

- **MHD blast** ($B_0 = 0.5$): With `B_0` $= 0.5$, $\theta = \pi/2$, $\phi = \pi/4$, and `r_0` $= 0.125$.

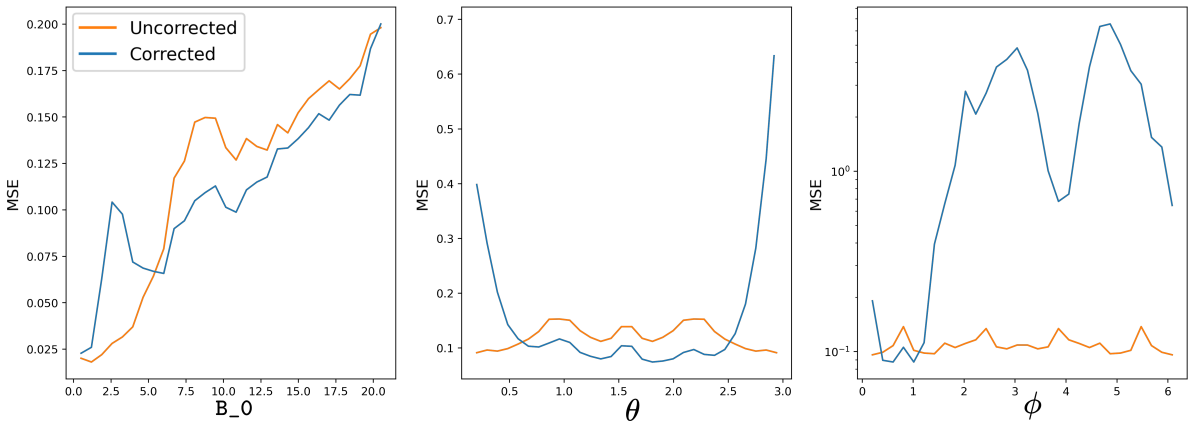


Figure 5.14: Loss parameter study of the MHD blast. The blast radius is fixed in all panels. From left to right, (i) the magnetic field strength is varied while the field direction is fixed at $\theta = \pi/2$ and $\phi = \pi/4$; (ii) the polar angle θ is varied while the magnetic field strength and azimuthal angle ($\phi = \pi/4$) are fixed; (iii) and the azimuthal angle ϕ is varied while the magnetic field strength and polar angle ($\theta = \pi/2$) are fixed, restricting the field direction to the x-y plane.

- **MHD blast** (B-field third quadarant): With `B_0` $= 10$, $\theta = \arccos(-1/\sqrt{3}) \approx 2.186$, $\phi = -3\pi/4$, and `r_0` $= 0.125$.
- **MHD blast** (Default): With `B_0` $= 10$, $\theta = \pi/2$, $\phi = \pi/4$, and `r_0` $= 0.125$.
- **OT vortex** ($\epsilon = 0.6$): With $\epsilon = 0.6$, `vortex_axis` $=$ z axis, and `parity` $= -1$.
- **OT vortex** (y-axis): With $\epsilon = 0.2$, `vortex_axis` $=$ y axis, and `parity` $= -1$.
- **OT vortex** (Default): With $\epsilon = 0.2$, `vortex_axis` $=$ z axis, and `parity` $= -1$.

The training loss evolution can be seen in Figure 5.15. Overall, the model converges across most of the 6 problems, with the exception of the MHD Blast with low magnetic field strength. For this problem, the loss evolution remains noisy and does not show convergence. This may be attributed to the difficulty of finding coherent updates in relation to the other 5 problems, as they all entail strong magnetic fields, thus, just by numbers, the optimisation goes in a strong magnetic field direction. Possible solutions include augmenting the number of problems which include low magnetic field regimes or an ensemble of models trained on different regimes that could later be orchestrated.

The parameter study is presented in Figure 5.16. Looking first at the MHD Blast

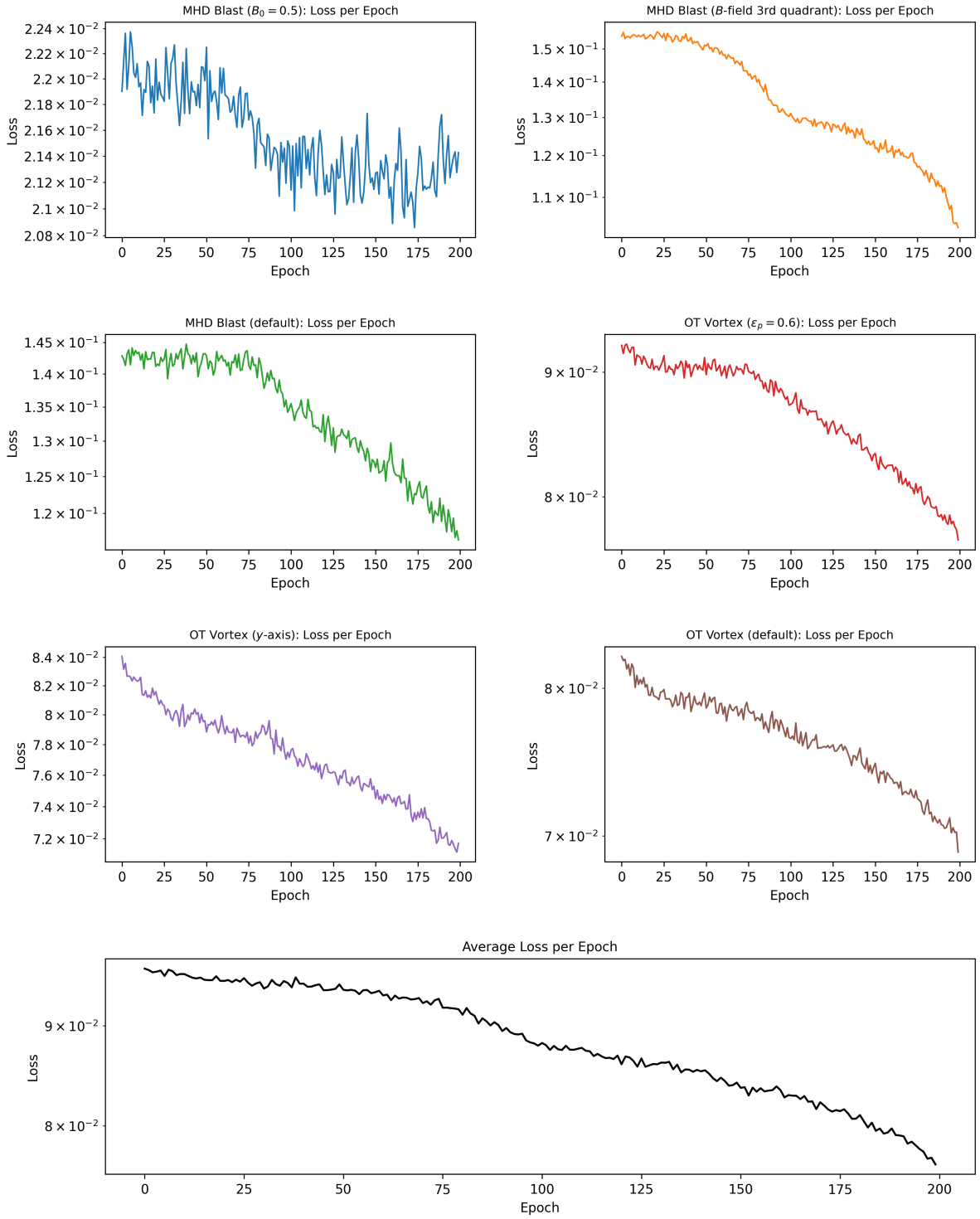
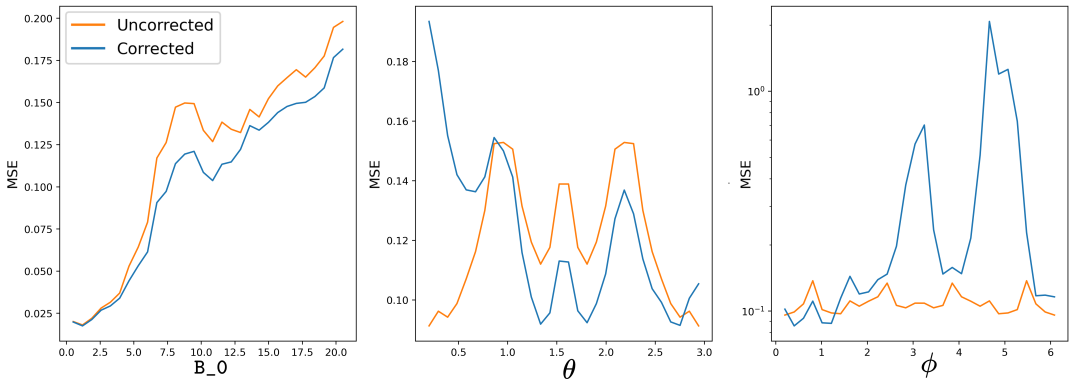
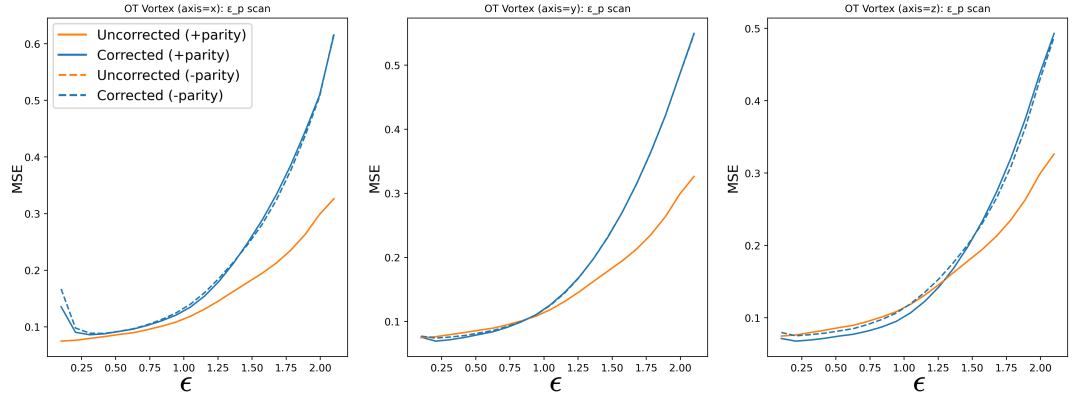


Figure 5.15: Training loss evolution of the SOL model trained on 6 different problems. The model was trained on 3 MHD Blasts and 3 OT vortices, each with different parameters.



(a) Loss parameter study of the MHD blast. The blast radius is fixed in all panels. From left to right, (i) the magnetic field strength is varied while the field direction is fixed at $\theta = \pi/2$ and $\phi = \pi/4$; (ii) the polar angle θ is varied while the magnetic field strength and azimuthal angle ($\phi = \pi/4$) are fixed; (iii) and the azimuthal angle ϕ is varied while the magnetic field strength and polar angle ($\theta = \pi/2$) are fixed, restricting the field direction to the x-y plane



(b) OT vortex parameter study. The ϵ parameter, which controls the perturbation amplitude in the plane perpendicular to the vortex axis, is varied. From left to right, different vortex orientations are studied by varying the axis perpendicular to the vortex plane, together with the effect of parity.

Figure 5.16: Generalization study performed on the model trained simultaneously on 3 MHD blasts and 3 OT vortices.

problem, the plots show improved performance mainly in the trained parameter range. We see improvements in the small valued B_0 region and around the two values of θ considered. We also observe improvement at the extreme $\theta = \pi$, whereas in the previous angle study it did explode in that regime. Still, we do not observe an improvement over the ϕ angle, which is to be expected as the model was only trained with 1 fixed $\phi = \pi/4$. Comparing the response over the θ range with the previous section, the model shows worse overall performance, this again can be attributed to a tradeoff in induced biases and performance.

Continuing on OT Vortex, the pronounced split between different parities observed in the 2 problem trained model is reduced. This, together with the improvements over the angle ranges in the MHD blast, may be an early sign of rotation invariance due to data augmentation. However, the evidence is not conclusive. Besides the model showcases improvement mainly for the set of trained parameters (axis y and z) whereas axis x remains worse than the low resolution reference.

5.5. Discussion

In this section solver-in-the-loop models were trained with multiproblem approaches and further studies into generalisation and data augmentation were done. It's important to remark the time and memory complexity that entails training any of these models as for low simulations the priors occupy substantial memory and strategies such as memory checkpointing have to be used. Thus, there's a high memory-time tradeoff; for instance, the last model shown, trained on 6 problems and 32^3 low resolution, was trained on 3 NVIDIA A100 GPUs with 40 GB of memory and took approximately 10 hours. Thus, SOL models might be useful only in scenarios where hundreds of simulations of 1 or 2 type problems with slightly different initial parameters are needed; still, in such a case, it is still to be compared with skipping the solver-in-the-loop and getting the simulations directly, due to the time it takes to train and finetune each of these models. Such an application could be Simulation Based Inference, where several simulations with different initial parameters are needed.

Commenting on the overall generalisation capabilities, early stages of generalisation were observed, although it is still unclear whether this was pure generalisation or interpolation. Strategies such as physically informed neural networks or gradient aggregation were used and proven to work. In future works, this could be expanded with further inductive biases and extra losses such as vorticity-based losses for turbulence. On the topic of turbulence, due to the chaoticness of the simulation, the early experiments done were unstable and eventually gradients exploded. This signifies the need for further generalisation strategies in future work.

One reference state per problem was used together with rather large simulation rollouts. This proved to work but also allowed for overfitting in phase space and underperformance on the unseen regions of the simulation. One strategy left to explore is to, instead of training on rather large rollouts and one reference state per problem, like the ones used, training on several small segments of a larger simulation. In such fashion, the dataset of initial states would be larger without largely trading off for memory time, as the simulation and gradient computation time would be shorter. It would also allow for more stable and faster loss convergence, and later in training, these time intervals could be increased to further motivate the model's performance in larger rollouts, allowing for both a quick and stable start and better performance on larger rollouts. An early approach of this strategy was tried with curriculum learning and the results were mixed. Here we propose taking smaller simulation times and more than once at an epoch, from different parts of the simulation.

On our multiproblem work we handpicked the problems to train on. Although this allows for quick experimentation, it does not allow for proper generalisation or big probes. The use of larger datasets with broader parameter ranges and stochastic decisions was tried, and in our initial experiments we found rather complex loss evolutions where the loss seemed noisy and nonconvergent. This approach, together with the previous, where smaller simulation segments would be used, might be fundamental in continuing the study of these models. This would allow models to be trained on several parameters and regimes without a large tradeoff in time or active memory.

In our work, only one architecture was considered; although it proved useful for the re-

duced experiments performed, our results and conclusions depend on it. One limitation of the model considered is its parameter agnosticness, especially with respect to simulation parameters. This makes the model unable to generalise through simulation parameters such as CFL constant, Riemann solvers or boundary conditions. Future works should explore other models in this direction. Examples of parameter-aware models are: a suite of models trained on different simulation parameters that could be orchestrated or models with specialised parameter-aware layers or blocks.

In conclusion solver-in-the-loop models are still to be further studied and expanded to be considered a proper tool in the MHD astrodynamical field. Nowadays, these models don't ensure proper generalisation nor performance across broad domains. Still, SOL architectures in theory pose as an intermediate machine learning application in the MHD field that, given sufficient proven performance, could become a functioning tool for large simulation rollouts, reducing time and memory costs on further ISM studies.